Latency Can Reverse the Ranking: Effective-Rate Evaluation of RIS Phase Optimization for Mobile Users

[Author names]

***Abstract*—Reconfigurable intelligent surface (RIS) phase optimizers are usually compared by the rate they reach on the channel they were given, but a mobile user receives a phase only after the optimizer has finished. We evaluate methods by this *effective rate*, with measured latencies, in a 3GPP indoor-office multi-user MISO downlink with a 32-element RIS and users walking at 1.5 m/s at 3 GHz. The methods with the highest rate at $t = 0$ need 0.9–5.9 s and end below a 2.5 ms single-user heuristic. Adam's effective rate peaks at 4–6 steps, and running it to 48 steps costs 14.0% (95% CI 12.5–15.5%). The optimum is governed by the step time relative to the Doppler frequency: as this ratio grows from 2.5×10^{-4} to 7.5×10^{-2}, the best step count falls from 96 to 2, so the conclusion depends on implementation speed. In a controller that re-optimizes periodically, the pilot overhead sets the update period, while a few warm-started steps per update remain best. On a static scenario set, a learned update rule beats tuned Adam at every equal step count, yet every learned configuration is dominated by a faster and better Adam configuration on measured latency. All results assume a precoder refreshed every frame, whose cost we measure.**

***Index Terms*—Reconfigurable intelligent surface, mobility, computation latency, effective rate, channel aging, deep unfolding, multi-user MISO.**

I. Introduction

RECONFIGURABLE intelligent surfaces (RIS) reshape the radio channel with a large number of passive, phase-controllable elements [1], [2], and prototypes and field trials have demonstrated them in practice [3], [4]. In a multi-user downlink the phases must be optimized jointly with the base-station (BS) precoder, which is a non-convex problem usually attacked with alternating optimization and semidefinite relaxation, block coordinate descent, fractional programming or WMMSE [2], [5]–[8], and, more recently, with learned policies [9].

These methods are almost always compared by the rate they reach on the channel they were given. That comparison is appropriate when the channel is static. When users move, it is not: a method delivers its phase configuration to the channel that exists when it finishes, not to the channel it was handed. A slow method answers a question about a channel that is already gone.

This paper makes that observation quantitative for a multi-user MISO downlink assisted by a wall-mounted RIS in a 3GPP TR 38.901 indoor-office scenario [10] with pedestrian users ($v = 1.5$ m/s, $f_c = 3$ GHz, reference coherence time $T_c = 0.423/f_d = 28.2$ ms). Every phase is computed on the channel at $t = 0$ and evaluated on a geometrically evolved channel at the method's own measured latency. We show when, and by how much, latency changes which method one should use.

[Funding statement, e.g., TÜBİTAK project number, or remove.]
[Authors] are with [Department, University, City, Turkey] (e-mail: [e-mail]).

A. Contributions

1) **Effective-rate evaluation.** We define the effective rate of a method as its minimum user rate on the channel that exists at its own completion time (Section IV) and measure it for single-user heuristics, Adam, steepest ascent with Armijo line search, coordinate descent and a long-run Adam optimizer (1000 steps from 3 random starts). Between Adam and Armijo, too, the per-step cost decides which is better. The two methods with the highest rate at $t = 0$ need 0.9–5.9 s, and their effective rates, optimistic values of our channel model, fall below that of a 2.5 ms heuristic.
2) **An intrinsic optimum in the step count.** For Adam, the $t = 0$ rate keeps increasing with the number of steps L, but the effective rate peaks at $L = 4$–6 (5.98 b/s/Hz) and declines beyond. Running to $L = 48$ costs 14.0% (95% CI 12.5–15.5%). The optimum also exists for Armijo steepest ascent and for every Adam step size we tried, and it moves to fewer steps for faster users and longer channel-acquisition delays, which we tie to the pilot overhead of cascaded channel estimation.
3) **A scaling law in the normalized step cost.** What sets the optimum is the step time relative to the channel's rate of change, $x = \Delta T f_d$. Sweeping x over two and a half decades moves the optimum from 96 steps to 2. The cost of running to convergence is 1.5% at the smallest x, peaks at about 21%, and falls again where little is left to gain. Because scaling the step time is equivalent to scaling the speed, this transfers the results to faster implementations and other speeds, approximately also to other carriers, and it states the condition under which convergence does *not* hurt.
4) **Periodic re-optimization.** In a controller that re-optimizes periodically and pays for the pilots of each update, the pilot duration sets the best update period (back-to-back to 80 ms), while a few steps per update remain best: with warm starts 2–8 steps, and warm starting gains up to 0.52 b/s/Hz.
5) **Ranking reversal between equal steps and equal milliseconds.** A learned (deep-unfolded) update rule

beats a step-size-tuned Adam at every budget $L \in \{1, 2, 4, 8, 16\}$ when both run the same number of steps (win rates 68–84% of test scenarios). Measured in five independent timing rounds on the same static scenario set, every learned configuration is dominated by an Adam configuration that is both faster and better. One learned step costs 2.7–4.0 Adam steps.

6) **Pricing the modelling assumption.** All effective rates assume that the BS precoder is refreshed from current pilots while only the RIS phase ages. If the precoder aged together with the phase, the rate at T_c would be 11.7–14.2× lower; refreshing it costs about 13× less than one Adam step.

All tables and figures are generated by scripts from stored measurement files, and the measurement traps that produced wrong numbers during this work are reported in the Appendix.

Section II reviews related work, Section III gives the system and channel model, Section IV defines the evaluation framework, Section V the measurement protocol, Section VI the results, and Section VII concludes.

II. Related Work

A. RIS Phase Optimization

Joint precoding and RIS phase design has been addressed with alternating optimization and semidefinite relaxation [2], beamforming under practical phase-dependent amplitude models [5], block-coordinate-descent designs for multicell systems [6], fractional programming for weighted sum-rate maximization [7] and WMMSE-based rate maximization [8]. These methods are evaluated by the objective value they reach; their run time, when discussed, is characterized by complexity order rather than by measured latency relative to the channel's coherence time.

Learning-based designs replace the iterative solver with a trained policy, for example deep reinforcement learning for joint precoding and phase design in multi-user MISO systems [9]. Algorithm unrolling, which turns the iterations of an optimizer into the layers of a trainable network [11], is the template for the learned update rule studied here; it has been applied to IRS beamforming to avoid the cost of matrix inversions in an iterative solver [12]. Learning-based designs are usually motivated by speed, yet that speed is rarely related to how fast the channel changes, and the classical solvers they are compared against are rarely tuned or timed under the same conditions.

B. Mobility and Outdated Channel Knowledge

High mobility has been addressed by mounting the surface on the vehicle so that a fast-fading channel becomes slow-fading [13]. The effect of channel aging [14] and of outdated channel state information [15] on RIS-aided systems has been analysed, and predictive beamforming, which acts on a forecast of the future channel, has been proposed for fast-moving links such as UAVs [16]. Channel aging has also been analysed for FDD massive MIMO [17], and the cost of imperfect channel knowledge at the transmitter is classical in MIMO broadcast channels [18]. Two-timescale designs update the RIS phases from slowly varying channel statistics and the BS precoder from instantaneous channel knowledge [19], which reduces how often the phases must be recomputed; this has also been developed for fast-changing channels, where the instantaneous channel knowledge used by the precoder is itself quickly outdated [20]. In these works the delay between channel acquisition and use is a system parameter. The optimizer's own run time, which differs from method to method and from iteration count to iteration count, is not part of the model.

C. Discrete Phases

Practical surfaces have a few bits of phase resolution [21]; a recent prototype uses 2-bit unit cells with microcontroller-based control for fast reconfiguration of the whole array [22]. This paper uses continuous phases; the latency dimension of discrete phase design is left to future work (Section VII).

D. Gap Addressed Here

To our knowledge, no prior RIS work (i) evaluates each method on the channel at its own measured completion time, (ii) shows that this creates an interior optimum in the iteration count of a standard optimizer, or (iii) compares a learned update rule with a tuned classical optimizer both at equal steps and at equal measured milliseconds and finds that the two comparisons rank them in opposite order.

III. System and Channel Model

A. Signal Model

A BS with a uniform linear array of $N_t = 8$ antennas at half-wavelength spacing serves $K = 4$ single-antenna users through a direct link and an RIS of $N = 32$ passive elements (an 8×4 planar array, half-wavelength spacing). With RIS phases $\boldsymbol{\theta} \in [0, 2\pi)^N$, $\mathbf{v} = e^{j\boldsymbol{\theta}}$, and precoder $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_K]$, user k receives

$$y_k = \left(\mathbf{h}_{d,k}^H + \mathbf{g}_k^H \operatorname{diag}(\mathbf{v})\mathbf{F}\right) \sum_{j=1}^{K} \mathbf{w}_j s_j + n_k, \qquad (1)$$

where $\mathbf{F} \in \mathbb{C}^{N \times N_t}$ is the BS–RIS channel, $\mathbf{g}_k^H \in \mathbb{C}^{1 \times N}$ the RIS–user channel, $\mathbf{h}_{d,k}^H \in \mathbb{C}^{1 \times N_t}$ the direct channel and $n_k \sim \mathcal{CN}(0, \sigma^2)$. Writing $\mathbf{h}_k^H(\boldsymbol{\theta})$ for the bracketed effective channel, the rate of user k is $R_k = \log_2(1 + \mathrm{SINR}_k)$ with

$$\mathrm{SINR}_k = \frac{|\mathbf{h}_k^H \mathbf{w}_k|^2}{\sum_{j \neq k} |\mathbf{h}_k^H \mathbf{w}_j|^2 + \sigma^2}. \qquad (2)$$

For a given phase the precoder is regularized zero forcing on the effective channel, with the MMSE regularizer $\lambda = K\sigma^2/P$ and total power P. The design objective is the minimum user rate,

$$\max_{\boldsymbol{\theta}} \min_{k} R_k(\boldsymbol{\theta}), \qquad (3)$$

which Adam and the learned update optimize through the smooth surrogate $-\frac{1}{\beta} \log \sum_k e^{-\beta R_k}$ with $\beta = 8$, evaluated with a log-sum-exp for numerical stability; coordinate descent and the long-run Adam reference maximize the hard minimum directly.

B. Geometry and Propagation

The room is $40 \times 20 \times 6$ m (Fig. 1). The RIS is mounted on a wall, centred at $(20, 0.1, 3)$ m and facing into the room; the BS is at $(20, 6, 4)$ m, in the panel's boresight and beyond its Fraunhofer distance. Users are dropped uniformly in the room at heights between 1 and 2 m. The carrier is $f_c = 3$ GHz, the bandwidth 20 MHz, the transmit power 20 dBm, the BS antenna gain 5 dBi and the receiver noise figure 7 dB.

All links follow the 3GPP TR 38.901 indoor-office model [10]: its path loss and LoS probability, a Ricean factor K_R drawn from its distribution, and log-normal shadowing with Gudmundson spatial correlation [23] at a 5 m decorrelation distance (TR 38.901 specifies 10 m for LoS and 6 m for NLoS indoor links). Non-LoS components are built from point scatterers placed in the room and shared by all links of a scenario. The cascaded BS–RIS–user amplitude follows the two-hop path-loss model of Tang et al. [24] with a $\cos^q$ element pattern. Because an RIS is deployed where the direct path is obstructed, the direct link is attenuated by an additional 40 dB.

C. Mobility

Users walk in a straight line at $v = 1.5$ m/s and are reflected at the walls. Over the observation window the scatterer positions, per-path complex gains, LoS states, Ricean factors, shadowing values and the BS–RIS channel $\mathbf{F}$ are held fixed; only the user positions move, and the channels $\mathbf{g}_k(t)$ and $\mathbf{h}_{d,k}(t)$ are recomputed from geometry at every time instant. Channel variation therefore comes from geometry rather than from a stochastic aging model. As a reference time scale we use the Doppler frequency $f_d = v f_c/c = 15$ Hz and the coherence time $T_c = 0.423/f_d = 28.2$ ms [25].

D. Known Modelling Limitations

We state the following limitations; the first two bias the results in a known direction.

- *Frozen scatterers.* Over seconds a walking user would see new scatterers, but the model keeps them. Over long horizons the modelled channel therefore changes less than a real one would, which *understates* the loss of slow methods. We treat latencies up to 200 ms as reliable and also report a stale floor, i.e., a $t = 0$ phase evaluated on an independent channel, as the true asymptote.
- *Independent per-antenna path gains.* The complex gains of the scattered paths are drawn independently for each BS antenna instead of being shared across the array. This adds antenna diversity, makes interference nulling easier, and so makes all rates optimistic.
- *Perfect channel knowledge.* Each method is given the exact channel at $t = 0$. Imperfect CSI is left to future work. The channel-acquisition delay is not part of the main results; Section VI-D adds it as a parameter.
- *Narrowband, far field.* The model is a single narrowband carrier, and the BS lies beyond the RIS's Fraunhofer distance. Wideband (OFDM) operation with beam squint and near-field operation of large surfaces are not covered.

IV. Latency-Aware Evaluation Framework

A. Effective Rate

Let a method A receive the channel $\mathcal{H}(0) = \{\mathbf{F}, \mathbf{g}_k(0), \mathbf{h}_{d,k}(0)\}$, return the phase $\boldsymbol{\theta}_A$, and need the wall-clock time τ_A to do so.

Definition 1 (Effective rate). The effective rate of A is its minimum user rate on the channel that exists when it finishes:

$$R_A^{\text{eff}} = \min_k R_k\big(\boldsymbol{\theta}_A;\, \mathcal{H}(\tau_A)\big), \tag{4}$$

where the precoder is recomputed from $\mathcal{H}(\tau_A)$.

The precoder is a closed-form digital computation that the BS can refresh every frame from pilots, whereas the RIS phase requires the iterative optimization whose latency is measured here. The same split between a fast precoder update and a slower phase update underlies two-timescale RIS designs [19]. Definition 1 therefore lets the phase age and keeps the beam fresh. Section VI-G measures how much this assumption is worth.

B. Gain–Aging Decomposition and the Intrinsic Optimum

For an iterative method run for L steps, write $r_0(L) = \min_k R_k(\boldsymbol{\theta}_L; \mathcal{H}(0))$ for its rate at $t = 0$, $\tau(L)$ for its latency and

$$R^{\text{eff}}(L) = r_0(L) - \Delta(L), \tag{5}$$

$$\Delta(L) = r_0(L) - \min_k R_k\big(\boldsymbol{\theta}_L; \mathcal{H}(\tau(L))\big), \tag{6}$$

so that $\Delta(L)$ is the rate lost to aging during the method's own run time. Additional steps raise r_0 with diminishing returns, while Δ grows with $\tau(L)$. An additional step pays off only while

$$r_0(L+1) - r_0(L) \;>\; \Delta(L+1) - \Delta(L), \tag{7}$$

and the effective rate peaks at the first L where this inequality fails. We call that point the *intrinsic optimum* $L^\star$. It is a property of the optimizer and its step size, the per-step latency and the speed of the channel together, not of the optimizer alone. A slower step or a faster user raises the right-hand side and moves $L^\star$ down. Equation (7) is a bookkeeping identity rather than a theorem: whether R^{eff} is unimodal is an empirical question, which we answer in Section VI-B.

C. Equal Steps versus Equal Milliseconds

Learned update rules are usually compared with classical optimizers at an equal number of steps. If one learned step costs c times as much as one classical step, an equal-step comparison gives the learned method c times the compute budget. We therefore also compare at equal measured time and use Pareto dominance.

Definition 2 (Dominance). Configuration B dominates configuration A if $\tau_B \le \tau_A$ and $r_{0,B} \ge r_{0,A}$, with one inequality strict, where r_0 is the rate at $t = 0$.

If B dominates A, there is no latency budget at which A is the better choice. A learned configuration that is dominated by some configuration of the classical optimizer is therefore never worth deploying, regardless of how it compares step for step.

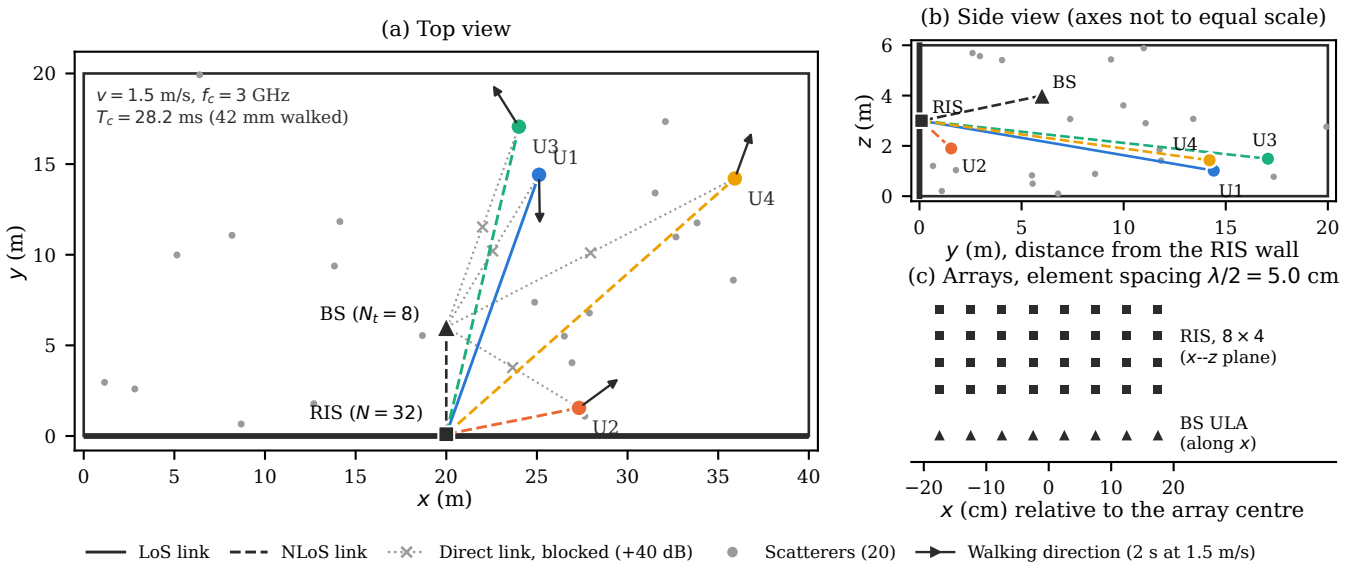


Fig. 1. The simulated scenario, drawn from the first scenario of the aging set (seed 11) exactly as the channel generator produced it. (a) Top view of the $40 \times 20 \times 6$ m office: the BS ($N_t = 8$) at $(20, 6, 4)$ m, the wall-mounted RIS ($N = 32$) centred at $(20, 0.1, 3)$ m, four users at 1–2 m height walking in straight lines at 1.5 m/s, and 20 point scatterers. Line style gives the LoS state drawn for each link from the 3GPP LoS probability. All direct links carry the 40 dB blockage. The arrows show 2 s of walking; within one coherence time a user moves only 42 mm. (b) Side view. (c) Element layouts of the RIS and the BS array.

D. Normalized Step Cost

Channel variation depends on how far a user moves in wavelengths, i.e., on $v\tau/\lambda = f_d\tau$. If an optimizer takes ΔT per step, its latency after L steps in these units is $L\,x$ with

$$x = \Delta T\, f_d. \tag{8}$$

In our model the users move $v\tau$ metres in time τ while everything else is fixed, so scaling the step time ΔT by a factor is exactly equivalent to scaling the speed v by the same factor at a fixed carrier; Section VI-C varies ΔT. The same x at a different carrier frequency is only approximately equivalent: at 28 GHz the user moves about 9 times fewer metres for the same x, which changes path loss, angles and the array's size in wavelengths.

E. Periodic Re-optimization

Definition 1 evaluates a phase at the instant it is delivered. A deployed controller instead repeats the optimization with period U: at each snapshot it spends T_{p} on pilots for the cascaded channel, during which no data is transmitted, then $L\Delta T$ on optimization, deploys the result, and waits for the next snapshot ($U \geq T_{\mathrm{p}} + L\Delta T$; *back-to-back* operation means $U = T_{\mathrm{p}} + L\Delta T$).

Definition 3 (Time-averaged effective rate). Let $\boldsymbol{\theta}(t)$ be the phase deployed at time t, and let $\mathcal{P}$ be the set of pilot intervals. The time-averaged effective rate over a window $\mathcal{W}$ is

$$\bar{R}^{\mathrm{eff}} = \frac{1}{|\mathcal{W}|} \int_{\mathcal{W}\setminus\mathcal{P}} \min_k R_k\big(\boldsymbol{\theta}(t);\, \mathcal{H}(t)\big)\, dt. \tag{9}$$

Each update can start from scratch (*cold start*) or from the phase currently deployed (*warm start*). The period trades pilot overhead against the age of the phase in use, and L trades the quality of each update against its delay.

V. Measurement Protocol

A. Scenario Sets

We use three scenario sets, all generated with seed 11, and never mix numbers from different sets in one table or comparison. Absolute rates differ between the sets, mainly because their numbers of scatterers differ.

- *Aging set* (Tables I, II and III, Figs. 2–4): 400 scenarios with walking users and 20 point scatterers per scenario, close to the 15–19 clusters of the 3GPP indoor-office model. The periodic re-optimization study regenerates the same 400 scenarios on a finer time grid; the generator's random draws do not depend on the time grid, and the SU-average rate at 0, 50, 100 and 200 ms reproduces the aging run.
- *Beam set* (Table VI, Fig. 6): a separate draw of 400 scenarios with walking users and the generator's default of 6 scatterers.
- *Static set* (Tables IV and V, Fig. 5): 6000 scenarios without mobility, split 70/15/15 into 4200 training scenarios and two pools of 900, from which 400 validation and 600 test scenarios are used. The generator is not prefix-stable, i.e., drawing n scenarios is not the same as taking the first n of a larger draw, so every trained model is evaluated with the sample count it was trained with.

B. Methods

SU-average and *SU-best-of* are closed-form single-user phase alignments; the latter keeps the best of $K+1$ candidates.

Adam optimizes the soft-min surrogate starting from SU-average. *Armijo* is steepest ascent on the same surrogate and from the same start, with a per-scenario backtracking line search: sufficient-increase constant 10^{-4}, halving, the step doubled after each accepted iteration and reset to the initial step a_0 after a failed search. *Coordinate descent* searches each element over a 16-level phase grid with the others fixed, for up to 8 sweeps. *Long-run Adam* is Adam with step size 0.05 for 1000 iterations from 3 random starts, keeping the best; both it and coordinate descent maximize the hard minimum. The *learned update* is a deep-unfolded network with L learned steps. Each step receives the gradient of the soft-min rate as input and exchanges information across users. A separate network is trained for each L, without labels, with the soft-min rate as the training objective.

Hyper-parameters on the static set. At every budget $L \leq 16$, Adam's step size is selected on the validation set from ten values in $[0.02, 2.0]$, together with the objective (soft-min or hard min). The selected step size is 0.75 with the soft-min objective at every L, an interior point of the grid. The larger budgets used only in the latency comparison ($L = 24, 32, 48, 64$) are tuned the same way on six values in $[0.2, 1.5]$. Comparing a learned rule with an untuned optimizer would overstate its advantage.

Hyper-parameters on the aging set. Adam uses step size 0.5; repeating the sweep with 0.25, 0.75 and 1.0 shows that 0.5 gives the highest peak. Armijo's initial step a_0 is chosen from $\{0.1, 0.3, 1, 3, 10\}$ by its best effective rate (3 is selected). Both choices use the same 400 scenarios on which results are reported, so they are robustness checks rather than held-out selections.

C. Timing

All latencies are wall-clock times for a single scenario on an Intel Xeon CPU at 2.1 GHz with one PyTorch thread, measured with `time.perf_counter`; the median over repetitions is kept, and each measurement is repeated in several rounds. These are latencies of an unoptimized Python implementation; Section VI-C shows how the results change for faster implementations.

- *Static set* (Table V). Adam and the learned update are timed from a shared, precomputed SU-average start, 25 repetitions per configuration, five rounds. The learned network gets three warm-up calls. Its input features are precomputed and excluded from the timing, which favours the learned method. The round-to-round spread is 1.15–1.60× (max/min). The timing script does not record the load average.
- *Aging set* (Table I). SU-average, SU-best-of, coordinate descent, long-run Adam and Adam with $L \in \{1, 2, 4, 8, 16, 32, 48\}$ are timed with one script, five rounds. The load average was 0.20 before the run and rose to 0.81 over the rounds, which is what the script's own busy thread produces. The Adam rows and all step sweeps use a per-step latency of 13.3/8 = 1.66 ms from an earlier measurement, which lies within the range measured here (1.49–2.02 ms per step; least-squares slope 1.50 ms).
- *Armijo versus Adam.* Armijo's cost per step depends on the number of backtracking evaluations, so both are timed for 6 steps (Armijo's optimum) on 20 different scenarios. The ratio of the medians is 1.26 (0.81–1.97 across scenarios), giving 2.09 ms per Armijo step. The load average was 0.18 when the script started; no other job was running. Because the ratio varies between scenarios, Section VI-B reports how the comparison depends on it.

D. Aging Sweeps and Statistics

The sweeps evaluate each phase on a fixed time grid and interpolate linearly at the method's latency plus the acquisition delay T_{ov}. Between grid points, and especially at 3 m/s where the grid is coarse relative to T_c, the effective rates are interpolated, and differences of a few thousandths of a b/s/Hz between neighbouring L are not resolved. Latencies beyond the last grid point (5.6 s) take its value.

Absolute rates vary strongly between scenarios: the 95% confidence intervals of the scenario means in Table I are ±0.17–0.24 b/s/Hz. Comparisons between methods or step counts are therefore made with a *paired* bootstrap over the 400 scenarios (2000 resamples), and we call a configuration *not significantly below the best* if its paired difference to the best is not significantly negative at the 95% level. Table II instead lists the step counts *within 0.5%* of the best, a simpler criterion. Best configurations are selected and tested on the same scenarios without a multiple-comparison correction; they should be read as estimates, with the stated ranges as their uncertainty.

E. Acquisition Delay and Pilot Overhead

We tie T_{ov} and T_{p} to the pilot overhead of cascaded channel estimation. With the scheme of [26], $K + N + \max(K - 1, \lceil (K-1)N/N_t \rceil) = 48$ pilot symbols suffice for $K = 4$, $N = 32$, $N_t = 8$; estimating each user separately with $N + 1$ reflection patterns would take $K(N+1) = 132$. With 5G NR numerology (14 OFDM symbols per slot of $15/\mathrm{SCS}$ ms [27]) and one OFDM symbol per pilot, 48 symbols last 3.43, 1.71 and 0.86 ms at 15, 30 and 60 kHz subcarrier spacing, and 132 symbols last 9.43, 4.71 and 2.36 ms. Feedback, estimation processing and the switching time of the RIS come on top and are not modelled. The range $T_{\mathrm{ov}} \in [0, 10]$ ms of Table II covers these cases, and Table III uses $T_{\mathrm{p}} \in \{0, 0.86, 1.71, 3.43\}$ ms.

F. Periodic Re-optimization

The controller of Definition 3 is simulated on a 0.5 ms grid over 200 ms, the horizon within which the mobility model is reliable. No data is transmitted during pilot intervals. We test update periods $U \in \{10, 20, 40, 80\}$ ms and back-to-back operation, with $L \in \{1, 2, 3, 4, 6, 8, 12, 16, 24, 48\}$ steps per update wherever $T_{\mathrm{p}} + L\Delta T \leq U$. The rate is averaged over 100–200 ms, after every policy has deployed at least once; the phase in use before the first deployment is SU-average.

G. Reproducibility

One script generates every table and figure in this paper from the stored result files. Re-running the aging sweeps reproduces the stored results bit for bit, and every new experiment asserts that it reproduces the stored means of the aging run before reporting anything. The project's separate audit script, which checks 56 assertions about the learning experiments of the underlying thesis, does not cover the numbers reported here.

VI. Results

A. Aging of a Fixed Phase and Effective Rates

In this scenario the RIS carries most of the signal. With the direct links blocked, the median per-user SNR (channel gain over all BS antennas times P/σ^2) is 20.0 dB for the direct link alone and 31.1 dB for the RIS path with the SU-average phase. Without the RIS, the minimum user rate is 1.744 b/s/Hz; a random phase gives 4.072 (Table I).

Fig. 2 fixes each method's phase at $t = 0$ and tracks the minimum user rate as users walk, with the precoder refreshed at every instant. Rates are flat for the first few milliseconds, drop sharply between roughly 5 and 30 ms, i.e., around T_c, and level off above the stale floor. The more a phase has been optimized, the more it has to lose: by $\tau = T_c$ long-run Adam has fallen from 7.117 to 5.473 b/s/Hz and Adam with $L = 8$ from 6.443 to 5.165, while SU-average has only moved from 4.624 to 4.503.

Table I evaluates each method at its own measured latency (Definition 1). Long-run Adam and coordinate descent have the best $t = 0$ rates but need 5.9 s and 0.88 s, by which time users have moved metres. Their effective rates lie beyond the 200 ms horizon where our model is reliable and are optimistic; the true values lie between the tabulated effective rate and the stale floor. Even the optimistic values are below that of SU-best-of, which takes 2.5 ms: in a paired comparison SU-best-of is ahead of coordinate descent by 0.297 b/s/Hz (95% CI 0.204–0.390) and of long-run Adam by 0.604 (0.415–0.805). Among the methods in Table I, Adam with $L = 8$ has the highest effective rate (5.849 b/s/Hz); the step sweep below shows that fewer steps do better still.

TABLE I

Rate at $t = 0$, latency, and effective rate on the aging set (400 scenarios, $v = 1.5$ m/s). Stale floor: $t = 0$ phase on an independent channel. Adam rows use 1.66 ms per step; other latencies are measured (Section V). 95% CIs of these scenario means are ± 0.17–0.24 b/s/Hz; comparisons in the text use paired differences. †Beyond the 200 ms reliable horizon; optimistic. *Beyond the last time-grid point (5.6 s); value at 5.6 s.

Method	t=0 (b/s/Hz)	Latency (ms)	Effective (b/s/Hz)	Stale (b/s/Hz)
No RIS (direct link)	1.744	–	–	–
Random phase	4.072	–	–	–
SU-average	4.624	0.42	4.623	4.315
SU-best-of	5.276	2.5	5.272	4.336
Adam, $L = 8$	6.443	13.3	5.849	4.326
Adam, $L = 12$	6.631	20.0	5.515	4.370
Adam, $L = 16$	6.754	26.6	5.324	4.381
Adam, $L = 20$	6.835	33.2	5.276	4.364
Coord. descent	6.925	880.8	4.975†	4.352
Long-run Adam	7.117	5851.7	4.667†*	4.423

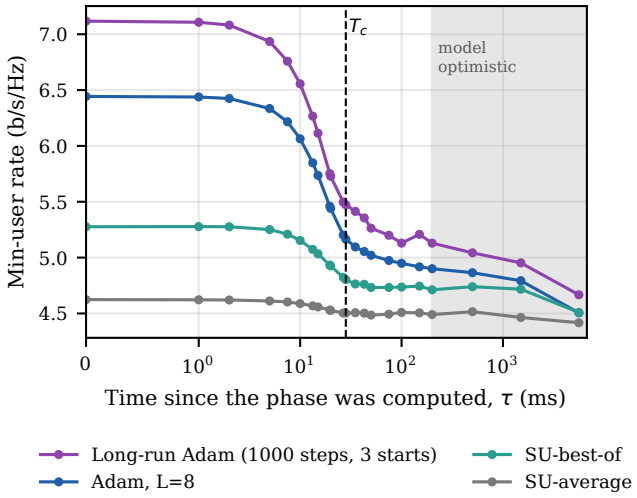


Fig. 2. Minimum user rate of a phase fixed at $t = 0$ as users walk at 1.5 m/s, with the precoder refreshed at every instant. Dashed line: $T_c = 28.2$ ms. Shaded: beyond 200 ms the frozen-scatterer model is optimistic.

B. The Intrinsic Optimum

Fig. 3 sweeps the number of optimizer steps, starting from SU-average (4.624 b/s/Hz); the point at $L = 0$ shows SU-best-of for reference. Adam's $t = 0$ rate grows monotonically with L, from 5.412 at $L = 1$ to 7.037 b/s/Hz at $L = 48$. The effective rate does not. It has a flat top between $L = 4$ and $L = 6$ (5.980, 5.982 and 5.970 b/s/Hz, at 6.7–10.0 ms); none of the three is significantly below the best, and every other step count is. At $L = 48$ (79.8 ms) the effective rate is 5.145 b/s/Hz, 14.0% (95% CI 12.5–15.5%) below the optimum, although the $t = 0$ rate there is 13.2% higher.

In terms of (7): going from $L = 4$ to $L = 6$ adds 0.188 b/s/Hz at $t = 0$, but the additional 3.3 ms of aging cost 0.198 b/s/Hz. Between $L = 6$ and $L = 48$ the effective rate declines; the only exception is a 0.006 b/s/Hz fluctuation between $L = 20$ and $L = 24$.

The optimum is not specific to Adam or to its step size. Repeating the Adam sweep with step sizes 0.25, 0.75 and 1.0 moves the best point to $L = 6$, 4 and 5, and running to $L = 48$ costs 11.5%, 13.7% and 11.3%. Armijo steepest ascent has its best effective rate at $L = 4$ (6.015 b/s/Hz at 2.09 ms per step). It gains more than Adam in the first steps (6.232 against 6.128 b/s/Hz at $L = 4$) but costs 1.26 times as much per step, and the two best effective rates are statistically indistinguishable: Armijo is ahead by 0.033 b/s/Hz (95% CI -0.001 to 0.064). Which of the two is better depends on the cost ratio. With Armijo's per-step cost set to 1.0, 1.2, 1.4, 1.6 and 2.0 times Adam's, its best effective rate is $+0.147$, $+0.050$, -0.013, -0.079 and -0.197 b/s/Hz relative to Adam's. Even between two classical optimizers, the per-step latency decides the ranking.

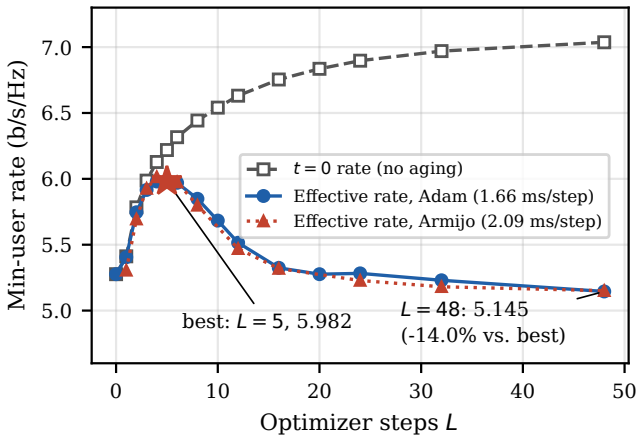


Fig. 3. Step sweep on the aging set. Adam's $t = 0$ rate increases monotonically, while the effective rate (rate at the method's own completion time) has a flat top at $L = 4$–6 and is 14.0% below its best value at $L = 48$. Armijo steepest ascent (2.09 ms per step) shows the same shape. $L = 0$: SU-best-of, for reference.

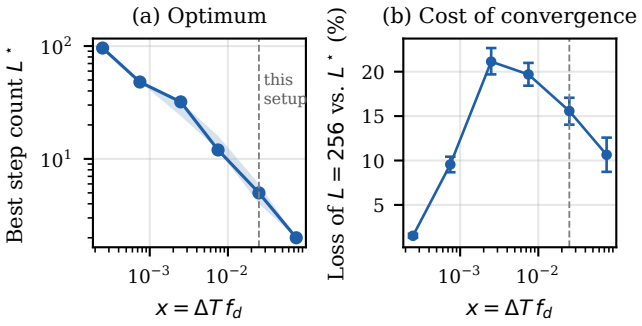


Fig. 4. Adam on the aging set with its per-step latency scaled from 0.01 to 3 times the measured value. (a) Best step count; shaded: step counts not significantly below the best. (b) Effective-rate loss of running to $L = 256$ instead of $L^\star$, with 95% paired bootstrap CIs. Dashed: this setup.

TABLE II
BEST NUMBER OF ADAM STEPS $L^\star$ ON THE GRID $L \in \{1, \ldots, 6, 8, 10, 12, 16, 20, 24, 32, 48\}$ AND, IN PARENTHESES, THE RANGE OF L WITHIN 0.5% OF THE BEST EFFECTIVE RATE, FOR DIFFERENT USER SPEEDS AND CHANNEL-ACQUISITION DELAYS T_{ov} (AGING SET). $^{\ddagger}T_{\mathrm{ov}} > T_c/2$: NOT INTERPRETED.

	Acquisition delay T_{ov} (ms)				
v (m/s), T_c	0	1	2	5	10
1, 42.3 ms	6 (6–8)	6 (6–8)	6 (5–8)	6 (5–6)	5 (4–6)
1.5, 28.2 ms	5 (4–6)	4 (4–5)	4 (4–5)	4 (3–4)	3 (3–4)
1.8, 23.5 ms	4 (4–5)	4 (3–4)	4 (3–4)	3 (3–4)	3 (3–4)
3, 14.1 ms	3 (3)	3 (3)	3 (2–3)	2 (2–3)	32‡ (32–48)

C. Dependence on the Step Cost

The latencies above are those of a single-threaded Python implementation; an optimized implementation would be much faster per step. Fig. 4 therefore scales Adam's per-step latency from 0.01 to 3 times the measured value, which moves the normalized step cost $x = \Delta T f_d$ of (8) from 2.5×10^{-4} to 7.5×10^{-2}; this setup sits at $x = 2.5 \times 10^{-2}$. By the equivalence of Section IV, the same axis describes users who are up to 100 times slower or 3 times faster.

The best step count falls steadily with x: 96 steps at $x = 2.5 \times 10^{-4}$, 48 at 7.5×10^{-4}, 24–32 at 2.5×10^{-3}, 12–16 at 7.5×10^{-3}, 4–6 here, and 2 at $x = 7.5 \times 10^{-2}$. Adam's $t = 0$ rate saturates near 100 steps (7.087 b/s/Hz at $L = 96$, 7.096 at $L = 256$), so at the smallest x the optimum is essentially convergence, and running to $L = 256$ costs only 1.5% (95% CI 1.3–1.8%). The cost of convergence peaks at about 21% for $x \approx 2.5 \times 10^{-3}$, where the optimizer is fast enough to gain much but convergence still takes long. It declines again for large x, where even the best policy has little left to gain. For $x \geq 2.5 \times 10^{-2}$, $L = 256$ lies beyond the 200 ms horizon, so its cost is if anything understated.

The claim that convergence hurts is thus a statement about a regime, not about RIS optimization in general. With a hundred-fold faster implementation at pedestrian speed, convergence is nearly free. The same implementation at vehicular speed, or at a millimetre-wave carrier, moves x in the opposite direction.

D. Sensitivity to Acquisition Delay and User Speed

The effective rate above charges each method only for its own run time. A deployed system also spends time on pilots, estimation and feedback before the optimizer starts, which for this system is between about 1 and 10 ms (Section V). We therefore add an acquisition delay T_{ov} to every latency and repeat the sweep at four user speeds (Table II). The direction is the one predicted by (7): a faster channel or a longer delay makes each additional step cost more aging, and the optimum moves to fewer steps. Without delay, the range of L within 0.5% of the best shifts from $L = 6$–8 at 1 m/s to $L = 3$ at 3 m/s, and at 1.5 m/s it shifts from $L = 4$–6 to $L = 3$–4 as T_{ov} grows from 0 to 10 ms. The best effective rate falls with T_{ov} at every speed. We do not interpret cells with $T_{\mathrm{ov}} > T_c/2$, where the channel has largely changed before optimization starts. The only such cell is 3 m/s with 10 ms; there all step counts give rates within 0.26 b/s/Hz, and the maximum falls at $L = 32$, which carries no meaning. In all other cells, running Adam to $L = 48$ costs 2.9–14.7% of the attainable effective rate.

The optimum is also tolerant of an error in the assumed speed. Choosing L for 1.5 m/s when users actually move at 1.8 m/s, or the reverse, costs less than 0.5% of the attainable effective rate. The largest loss in our grid, 5.7%, occurs when L is chosen for 1 m/s and users move at 3 m/s.

E. Periodic Re-optimization

Table III simulates the controller of Definition 3 and reports, for each pilot duration, the best combination of update period U and steps per update L. Pilot overhead is what sets the period. If pilots were free, back-to-back updates would be best, with two warm-started steps (6.36 b/s/Hz) or three cold-started ones. With 0.86 ms of pilots per update, the best period is 10–20 ms; with 1.71 ms it is 20 ms for warm starts; with 3.43 ms the best policies update every 80 ms, or back-to-back with 48 steps, which amounts to the same thing. Short back-to-back cycles then spend most of their time on pilots.

TABLE III
PERIODIC RE-OPTIMIZATION (DEFINITION 3, AGING SET): BEST UPDATE PERIOD U (MS; B2B = BACK-TO-BACK) AND STEPS PER UPDATE L, WITH THE TIME-AVERAGED EFFECTIVE RATE OVER 100–200 MS (B/S/HZ), FOR PILOT DURATIONS T_{p} WITHOUT DATA TRANSMISSION. WARM−COLD: PAIRED DIFFERENCE OF THE TWO BEST POLICIES (B/S/HZ); §NOT SIGNIFICANT AT 95%.

	Cold start			Warm start			
T_{p} (ms)	U	L	rate	U	L	rate	warm−cold
0	b2b	3	5.83	b2b	2	6.36	+0.52
0.86	20	4	5.20	10	3	5.42	+0.22
1.71	b2b	48	5.09	20	3	5.23	+0.15
3.43	b2b	48	4.98	80	8	4.99	+0.01§

TABLE IV
EQUAL NUMBER OF STEPS, STATIC TEST SET (600 SCENARIOS). ADAM STEP SIZE AND OBJECTIVE SELECTED ON 400 VALIDATION SCENARIOS AT EACH L.

L	Learned (b/s/Hz)	Adam (b/s/Hz)	Ratio	Diff. −2 s.e. (b/s/Hz)	Wins (%)
1	4.460	4.381	1.018	+0.061	74.8
2	4.879	4.693	1.040	+0.160	83.2
4	5.285	5.025	1.052	+0.228	83.7
8	5.593	5.380	1.040	+0.184	83.2
16	5.800	5.739	1.011	+0.039	68.3

What does not change is that few steps per update suffice. With warm starts the best policy uses 2, 3, 3 and 8 steps for the four pilot durations. In every row, some policy with at most 8 steps per update is not significantly below the best one, and with warm starts and pilots of up to 1.71 ms the best policy is significantly better than every policy with more steps. Warm starting gains 0.52, 0.22 and 0.15 b/s/Hz for the three shorter pilot durations; with 3.43 ms pilots updates are so rare that the gain vanishes.

In short, periodic operation confirms the single-update result: the channel-acquisition cost decides how often to update, and the effective-rate trade-off decides how much to compute per update, which is little.

F. Learned versus Classical: Equal Steps and Equal Milliseconds

This comparison is made on the static set, without mobility, and with $t = 0$ rates. It isolates the question of compute efficiency, which the previous subsections show to be what matters once latency enters. The learned model was not evaluated on the aging set.

Table IV compares the learned update rule with tuned Adam at equal step counts on the static test set; both start from SU-average (3.539 b/s/Hz). The learned rule is ahead at every budget, the paired difference minus two standard errors is positive at every budget, and it wins on 68–84% of test scenarios. On a step axis, the learned rule is the better optimizer.

Steps, however, are not equally expensive. One learned step takes 4.6–6.5 ms, one Adam step 1.60–1.71 ms, so a learned step costs 2.7–4.0 Adam steps. Table V and Fig. 5(b) place both methods on the measured latency axis. Every learned configuration is dominated by an Adam configuration that is both faster and better. This holds in each of the five timing rounds separately, although not always by the configuration listed: for learned $L = 16$, Adam with $L = 48$ is slower in two rounds, and Adam with $L = 32$ (6.013 b/s/Hz, at most 62.97 ms) dominates in every round. For example, the learned rule with $L = 4$ needs 18.41 ms for 5.285 b/s/Hz, while Adam with $L = 8$ needs 13.48 ms for 5.380 b/s/Hz. The ranking from the equal-step comparison reverses completely.

TABLE V
EQUAL MILLISECONDS: EACH LEARNED CONFIGURATION AND THE BEST ADAM CONFIGURATION THAT DOMINATES IT (LOWER MEDIAN LATENCY AND HIGHER RATE). MEDIANS OVER FIVE ROUNDS. IN EVERY ROUND EACH LEARNED CONFIGURATION IS DOMINATED BY AT LEAST ONE ADAM CONFIGURATION.

Learned update			Dominating Adam		
L	ms	b/s/Hz	L	ms	b/s/Hz
1	5.16	4.460	2	3.20	4.693
2	12.95	4.879	4	6.49	5.025
4	18.41	5.285	8	13.48	5.380
8	36.76	5.593	16	27.31	5.739
16	83.80	5.800	48	78.77	6.107

Three remarks bound this result. First, aging was not included; since the learned configuration is the slower one in every pair, aging would be expected to widen rather than close the gap, but we have not measured this. Second, the learned timing excludes feature computation, so the latency gap is if anything understated. Third, the cost ratio is a property of this implementation: on hardware that accelerates the network's matrix products more than Adam's step, the reversal could weaken.

G. What the Fresh-Beam Assumption Is Worth

All results above assume that the BS precoder is recomputed from the current channel while only the RIS phase ages (Section IV). Table VI and Fig. 6 measure the alternative in which the precoder ages too, on the separate beam set (Section V), whose absolute rates are not comparable with Table I. Both curves use the same $t = 0$ phase. With a fresh beam the rate decays moderately (Adam with $L = 8$: from 5.342 to 4.358 b/s/Hz at T_c); with a stale beam it collapses within tens of milliseconds, and at T_c it is 11.7–14.2× lower than with a fresh beam, for every phase choice. The assumption therefore matters, and it must hold in a deployment for the effective rates above to apply: the BS has to refresh its precoder at least once per coherence time. Refreshing it is cheap: single-threaded, computing the effective channel and the regularized ZF precoder takes a median of 0.12 ms, about 13× less than a single Adam step (8.0–15.0× across five rounds). We have not included this time in the effective rates. This is the usual frame-by-frame operation of a multi-antenna BS, but it is an assumption of our results rather than a finding.

VII. CONCLUSION

When users move, an RIS phase configuration is worth what it achieves on the channel that exists when it is delivered. Evaluated this way on a 3GPP indoor-office multi-user MISO scenario with pedestrian users, the methods with the best rate

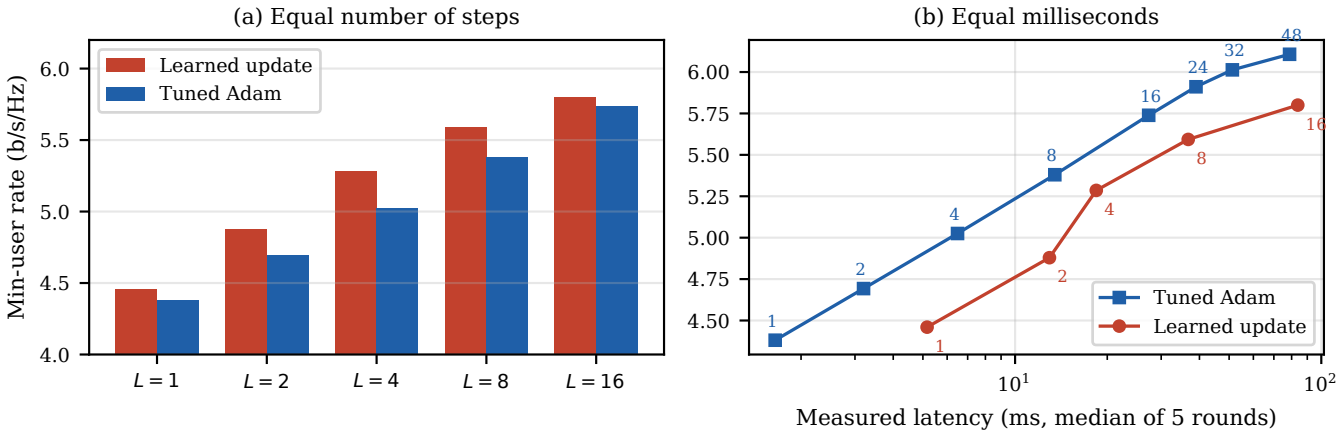


Fig. 5. Learned update rule versus tuned Adam on the static test set. (a) At an equal number of steps, the learned rule is ahead at every budget. (b) On the measured latency axis (numbers: steps), every learned configuration lies below and to the right of the Adam curve, i.e., it is dominated.

TABLE VI
FRESH VERSUS STALE PRECODER AT $\tau = T_c = 28.2$ MS, PHASE FIXED AT $t = 0$ IN BOTH (BEAM SET, 400 SCENARIOS).

Phase chosen at $t = 0$	Fresh beam (b/s/Hz)	Stale beam (b/s/Hz)	Ratio
SU-average	3.586	0.306	11.7×
SU-best-of	3.924	0.305	12.9×
Adam, $L = 8$	4.358	0.307	14.2×

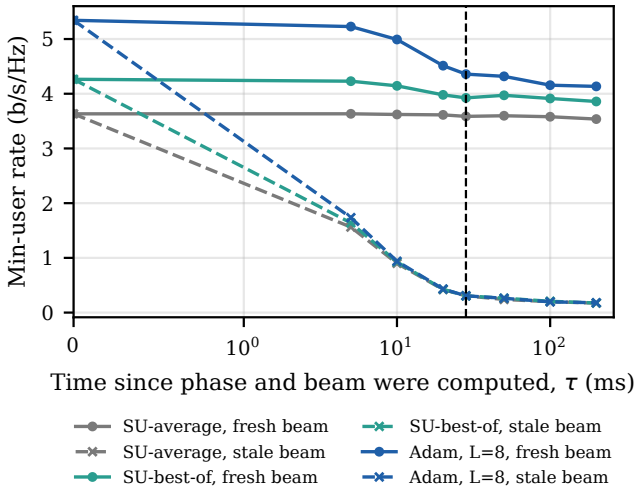


Fig. 6. Same $t = 0$ phase evaluated with a precoder recomputed from the current channel (fresh) or kept from $t = 0$ (stale). Dashed line: T_c.

at $t = 0$ take 0.9–5.9 s and end below a 2.5 ms single-user heuristic, and the number of optimizer steps has an interior optimum: Adam's effective rate peaks at 4–6 steps, and running it to $L = 48$ costs 14.0%. The optimum is governed by the normalized step cost $x = \Delta T f_d$. It moves from 96 steps to 2 as x grows over two and a half decades, so whether convergence hurts depends on how fast the optimizer runs relative to how fast the channel changes. In a controller that re-optimizes periodically, the pilot overhead of each update sets how often to update, while a few warm-started steps per update remain best. A learned update rule that beats a tuned Adam at every equal step count is dominated by Adam at every measured latency. Latency does not always decide the ranking, but in the regime quantified here it reverses it.

A. Practical Implications

- Choose the number of steps from $x = \Delta T f_d$ and the acquisition delay, not from convergence. In the setup studied here, four to six steps (7–10 ms) are best for a single update, and two to eight warm-started steps per update in periodic operation.
- Choose the update period from the pilot overhead, warm-start each update, and keep each update short.
- Compare methods on measured latency, with the classical baseline's hyper-parameters tuned at each budget, and report dominance rather than equal-step wins.
- Measure latency only on an idle machine, report it relative to $1/f_d$, and check that the verdict holds in every repeated round.

B. Limitations and Future Work

The channel model keeps scatterers fixed, so effective rates beyond about 200 ms are optimistic, and scattered-path gains are drawn independently per BS antenna, which makes all rates optimistic. Channel knowledge is perfect apart from the acquisition delay and pilot overhead we add, RIS switching time is not modelled, and the model is narrowband and far-field, with one geometry, $N = 32$ and $f_c = 3$ GHz; the normalized step cost carries the aging results to other speeds and carriers, but not path loss or larger surfaces. The learned update rule was compared with Adam only on the static set; its effective rate under mobility, and the cost ratio on hardware other than a single CPU thread, remain to be measured. Among classical methods we evaluated Adam, Armijo steepest ascent, coordinate descent and a long-run Adam reference, and found that even the ranking of Adam and Armijo depends

on their per-step cost; semidefinite relaxation [2], WMMSE-type algorithms [8] and manifold optimization were not timed. The effective-rate results also rely on the BS refreshing its precoder at least once per coherence time. Natural extensions are imperfect CSI, where a learned rule trained on noisy channels might outperform a classical optimizer that overfits the estimate, with clean-trained and noise-trained models reported separately; discrete phases (1–3 bits), including the latency of discrete optimizers; and wideband, near-field and vehicular operation.

APPENDIX

Three errors occurred during this work. Each produced a wrong number, and each led to a rule in the protocol of Section V.

Timing under load. A timing of Adam with $L = 8$ taken while a training job was running gave 29.70 ms; on the idle machine the same configuration measured 17.48 ms. The slowdown was not neutral: it hit Adam harder than the learned network and put a learned configuration on the latency–rate front. We now time only on an idle machine and report a dominance verdict only if it holds in every round.

A number without a script. An earlier draft stated that a fresh beam is worth "17×" a stale one. The figure had been computed once, interactively, and could not be regenerated; measured with a script, it is 11.7–14.2× (Table VI).

Stale derived documents. A human-readable summary had not been regenerated after a latency re-measurement and still listed older timings. The dominance verdict was the same, but the milliseconds were not. Tables are therefore generated directly from the result files, never copied from summaries.

ACKNOWLEDGMENT

[Acknowledgment text, e.g., advisor and computing resources, or remove this section.]

DATA AND CODE AVAILABILITY

The scripts that generate every table and figure, and the stored measurement files they read, are available at [public repository URL or "upon request"].

REFERENCES

[1] E. Basar, M. D. Renzo, J. D. Rosny, M. Debbah, M.-S. Alouini, and R. Zhang, "Wireless communications through reconfigurable intelligent surfaces," *IEEE Access*, vol. 7, pp. 116 753–116 773, 2019.

[2] Q. Wu and R. Zhang, "Intelligent reflecting surface enhanced wireless network via joint active and passive beamforming," *IEEE Transactions on Wireless Communications*, vol. 18, no. 11, pp. 5394–5409, 2019.

[3] L. Dai, B. Wang, M. Wang, X. Yang, J. Tan, S. Bi, S. Xu, F. Yang, Z. Chen, M. D. Renzo, C.-B. Chae, and L. Hanzo, "Reconfigurable intelligent surface-based wireless communications: Antenna design, prototyping, and experimental results," *IEEE Access*, vol. 8, pp. 45 913–45 923, 2020.

[4] X. Pei, H. Yin, L. Tan, L. Cao, Z. Li, K. Wang, K. Zhang, and E. Björnson, "RIS-aided wireless communications: Prototyping, adaptive beamforming, and indoor/outdoor field trials," *IEEE Transactions on Communications*, vol. 69, no. 12, pp. 8627–8640, 2021.

[5] S. Abeywickrama, R. Zhang, Q. Wu, and C. Yuen, "Intelligent reflecting surface: Practical phase shift model and beamforming optimization," *IEEE Transactions on Communications*, vol. 68, no. 9, pp. 5849–5863, 2020.

[6] C. Pan, H. Ren, K. Wang, W. Xu, M. Elkashlan, A. Nallanathan, and L. Hanzo, "Multicell MIMO communications relying on intelligent reflecting surfaces," *IEEE Transactions on Wireless Communications*, vol. 19, no. 8, pp. 5218–5233, 2020.

[7] H. Guo, Y.-C. Liang, J. Chen, and E. G. Larsson, "Weighted sum-rate maximization for reconfigurable intelligent surface aided wireless networks," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3064–3076, 2020.

[8] H. Choi, A. L. Swindlehurst, and J. Choi, "WMMSE-based rate maximization for RIS-assisted MU-MIMO systems," *IEEE Transactions on Communications*, vol. 72, no. 8, pp. 5194–5208, 2024.

[9] C. Huang, R. Mo, and C. Yuen, "Reconfigurable intelligent surface assisted multiuser MISO systems exploiting deep reinforcement learning," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 8, pp. 1839–1850, 2020.

[10] 3GPP, "Study on channel model for frequencies from 0.5 to 100 GHz," 3rd Generation Partnership Project (3GPP), Technical Report TR 38.901, V16.1.0 (Release 16), Dec. 2019.

[11] V. Monga, Y. Li, and Y. C. Eldar, "Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing," *IEEE Signal Processing Magazine*, vol. 38, no. 2, pp. 18–44, 2021.

[12] Y. Liu, Q. Hu, Y. Cai, G. Yu, and G. Y. Li, "Deep-unfolding beamforming for intelligent reflecting surface assisted full-duplex systems," *IEEE Transactions on Wireless Communications*, vol. 21, no. 7, pp. 4784–4800, 2022.

[13] Z. Huang, B. Zheng, and R. Zhang, "Transforming fading channel from fast to slow: Intelligent refracting surface aided high-mobility communication," *IEEE Transactions on Wireless Communications*, vol. 21, no. 7, pp. 4989–5003, 2022.

[14] A. Papazafeiropoulos, I. Krikidis, and P. Kourtessis, "Impact of channel aging on reconfigurable intelligent surface aided massive MIMO systems with statistical CSI," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 1, pp. 689–703, 2023.

[15] Y. Zhang, J. Zhang, M. D. Renzo, H. Xiao, and B. Ai, "Reconfigurable intelligent surfaces with outdated channel state information: Centralized vs. distributed deployments," *IEEE Transactions on Communications*, vol. 70, no. 4, pp. 2742–2756, 2022.

[16] C. Liu, W. Yuan, Z. Wei, X. Liu, and D. W. K. Ng, "Location-aware predictive beamforming for UAV communications: A deep learning approach," *IEEE Wireless Communications Letters*, vol. 10, no. 3, pp. 668–672, 2021.

[17] R. Chopra, C. R. Murthy, H. A. Suraweera, and E. G. Larsson, "Performance analysis of FDD massive MIMO systems under channel aging," *IEEE Transactions on Wireless Communications*, vol. 17, no. 2, pp. 1094–1108, 2018.

[18] N. Jindal, "MIMO broadcast channels with finite-rate feedback," *IEEE Transactions on Information Theory*, vol. 52, no. 11, pp. 5045–5060, 2006.

[19] M.-M. Zhao, Q. Wu, M.-J. Zhao, and R. Zhang, "Intelligent reflecting surface enhanced wireless networks: Two-timescale beamforming optimization," *IEEE Transactions on Wireless Communications*, vol. 20, no. 1, pp. 2–17, 2021.

[20] Y. Cao, T. Lv, and W. Ni, "Two-timescale optimization for intelligent reflecting surface-assisted MIMO transmission in fast-changing channels," *IEEE Transactions on Wireless Communications*, vol. 21, no. 12, pp. 10 424–10 437, 2022.

[21] Q. Wu and R. Zhang, "Beamforming optimization for wireless network aided by intelligent reflecting surface with discrete phase shifts," *IEEE Transactions on Communications*, vol. 68, no. 3, pp. 1838–1851, 2020.

[22] Y. Zhao, Y. Feng, A. M. Ismail, Z. Wang, Y. L. Guan, Y. Guo, and C. Yuen, "2-bit RIS prototyping enhancing rapid-response space-time wavefront manipulation for wireless communication: Experimental studies," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 4885–4901, 2024.

[23] M. Gudmundson, "Correlation model for shadow fading in mobile radio systems," *Electronics Letters*, vol. 27, no. 23, pp. 2145–2146, 1991.

[24] W. Tang, M. Z. Chen, X. Chen, J. Y. Dai, Y. Han, M. D. Renzo, Y. Zeng, S. Jin, Q. Cheng, and T. J. Cui, "Wireless communications with reconfigurable intelligent surface: Path loss modeling and experimental measurement," *IEEE Transactions on Wireless Communications*, vol. 20, no. 1, pp. 421–439, 2021.

[25] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Prentice Hall, 2002.

[26] Z. Wang, L. Liu, and S. Cui, "Channel estimation for intelligent reflecting surface assisted multiuser communications: Framework, algorithms, and analysis," *IEEE Transactions on Wireless Communications*, vol. 19, no. 10, pp. 6607–6620, 2020.

[27] 3GPP, “NR; physical channels and modulation,” 3rd Generation Partnership Project (3GPP), Technical Specification TS 38.211 (Release 16), 2020.